# Mini Project 3: AI-Assisted Cybersecurity Threat Model

| | |
|---|---|
| **Student name** | Allamsetti Venkata Soumya Sri |
| **Roll number / Student Code** | STU-93A9D6 |
| **Course / Group** | Cyber Security / G17 |
| **Platform** | SURE ProEd |
| **Host OS** | Windows 11 |
| **Date** | 24/10/2026 |

## 1. Aim

To analyse cybersecurity threats in the ThreatLens AI website security risk assessment and threat modeling system using the STRIDE methodology and AI-assisted recommendations. The report uses the supplied runtime screenshots, source code implementations, and automated test outputs as evidence and does not treat AI recommendations as proof of implemented controls.

## 2. Objectives

- Review the multi-tier ThreatLens AI architecture encompassing the browser UI, Flask web service, passive auditing engine, anti-SSRF validator, and SQLite database.
- Analyse four key threats affecting transport layer security, HTTP security header execution context, frame clickjacking boundaries, and request validation / audit-record integrity.
- Classify each threat systematically using the STRIDE threat-modeling methodology.
- Use AI to explain attack scenarios, impacts, mitigations, limitations and verification evidence.
- Compare original baseline mitigations with AI-assisted recommendations.
- Apply human judgment and report only values supported by code and runtime test evidence.

## 3. Introduction

Threat modeling identifies how a system could be attacked and what protections may reduce the risk. In this practical, the ThreatLens AI defensive ecosystem was modeled to evaluate website configurations, detect missing defenses, and map observable findings to structured STRIDE threat scenarios. The architecture bridges safe client-side observation and analytical backend services. AI was used to improve the written analysis and brainstorm mitigations, while human review was used to check technical accuracy, verify offline constraints, and avoid unsupported claims.

### 3.1 Background: Website Security Risk Assessment & Threat Modeling System

The model contains an Internet-facing context (untrusted target websites, cleartext port 80 endpoints, and redirection paths), an internal auditing engine context (anti-SSRF pre-validator, DNS resolver, TLS socket inspector, and heuristic security header analyzer), a Flask application service, and a persistent SQLite database. The anti-SSRF validator resolves the hostname with socket.getaddrinfo before any request is made and blocks loopback, RFC 1918 private, link-local, reserved and cloud-metadata addresses, as well as .local and .internal hostnames. The tool runs locally without reliance on paid external cloud APIs.

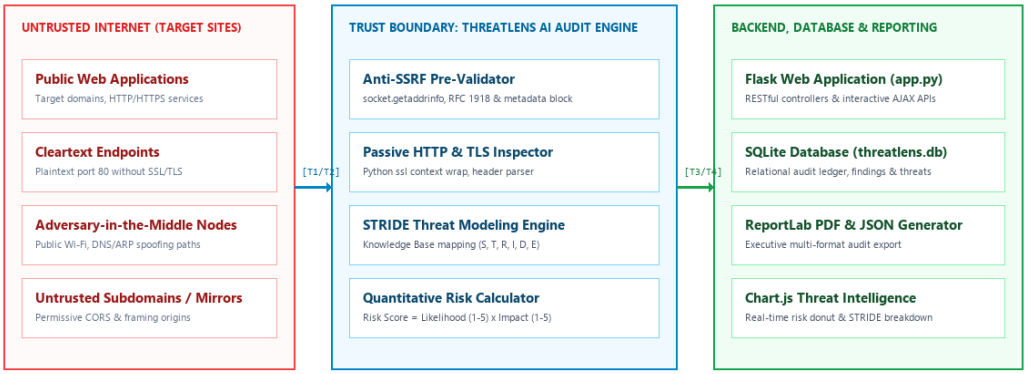



*Figure 1: ThreatLens AI Multi-Tier System Architecture and STRIDE Boundary Diagram.*

## 4. STRIDE Threat Modeling

STRIDE is a standardized threat-modeling framework covering six distinct threat categories: Spoofing, Tampering, Repudiation, Information Disclosure, Denial of Service, and Elevation of Privilege. All six categories are defined in the table below. The four threat scenarios analysed in this report map to Spoofing, Tampering, Information Disclosure, Repudiation and Elevation of Privilege. UI clickjacking is analysed as Elevation of Privilege, with UI redressing as the attack mechanism. The ThreatLens knowledge base generates Spoofing, Tampering, Denial of Service, Information Disclosure and Elevation of Privilege records but does not generate Repudiation records, so Repudiation is assessed manually in Threat 4.

| STRIDE | Full form | Meaning in this practical |
|---|---|---|
| S | Spoofing | An attacker impersonates a legitimate web application via expired or untrusted TLS certificates. |
| T | Tampering | An attacker intercepts unencrypted HTTP traffic, performs SSL-stripping, injects malicious scripts, or modifies stored records. |
| R | Repudiation | A user or attacker denies an action because reliable audit-log evidence is missing. |
| I | Information Disclosure | Server banners, cookie flags, URL query parameters or internal resources are exposed. |
| D | Denial of Service | A web service or audit tool becomes unresponsive due to infinite redirect loops or oversized response streams. |
| E | Elevation of Privilege | An adversary leverages UI redressing (clickjacking) or MIME confusion to trick authenticated users into executing unauthorized privileged actions. |

## 5. Tools and Technologies Used

- Python 3.x & Flask: Backend application server providing security check controllers, risk scoring APIs, and threat classification matrix.
- SQLite 3: Lightweight relational database storing persistent assessment summaries, granular findings, and threat models.
- Standard Socket & SSL: Low-level Python libraries for passive TLS handshake, cipher inspection, and certificate validation.
- Anti-SSRF Protection Engine: Validator enforcing pre-connection DNS resolution and blocking private IP subnets.
- BeautifulSoup4: Static HTML parser for detecting unencrypted HTTP asset references on secure HTTPS contexts.
- ReportLab & Chart.js: PDF report generation and client-side threat intelligence chart rendering.
- pytest: Automated unit and integration tests.
- **AI assistant: Antigravity** - analytical recommendation, attack scenario expansion, and mitigation explanation support.

- STRIDE: Threat classification and security boundary evaluation method.

## 6. System Configuration and Model Evidence

| Component | Details |
|---|---|
| Threat-modeling framework | Microsoft STRIDE & Quantitative Risk Scoring (Likelihood x Impact = 1 to 25) |
| Severity bands | 1-4 Low, 5-9 Medium, 10-16 High, 17-25 Critical |
| Model | ThreatLens AI Multi-Tier Website Security Risk Assessment Architecture |
| Client UI Component | Web Frontend (HTML5, Vanilla CSS Cyber Theme, JavaScript, Chart.js) |
| Application component | Python Flask Web Application (REST API / Assessment Controller) |
| Security Engine | Passive Auditing Engine (security_checks.py) & Anti-SSRF Pre-Validator |
| Database component | SQLite (threatlens.db with parameterized SQL queries) |
| Threat Feed Source | STRIDE Heuristic Threat Knowledge Base (threat_model.py) & OWASP Top 10 Guidelines |
| Evidence source | Source code files, Web dashboard, results screen, and automated pytest execution logs |

## 7. Methodology and Practical Procedure

1. Opened and structured the ThreatLens AI project files under Flask architecture.
2. Implemented the anti-SSRF validator with socket.getaddrinfo to block loopback, RFC 1918, and cloud metadata targets.
3. Implemented safe passive security checks for HTTPS redirection, TLS certificate validity, and response headers.
4. Built the HTTP security header inspector evaluating CSP, HSTS, X-Content-Type-Options, and Frame Protection.
5. Developed the Flask web controller and SQLite database schema with parameterized queries.
6. Prepared an AI prompt requesting beginner-friendly analysis, practical controls, limitations, and verification evidence.
7. Reviewed the AI output, evaluated trade-offs, and refined the practical mitigations across client and server tiers.
8. Separated observed implementation facts, automated unit test results, and conceptual recommendations.

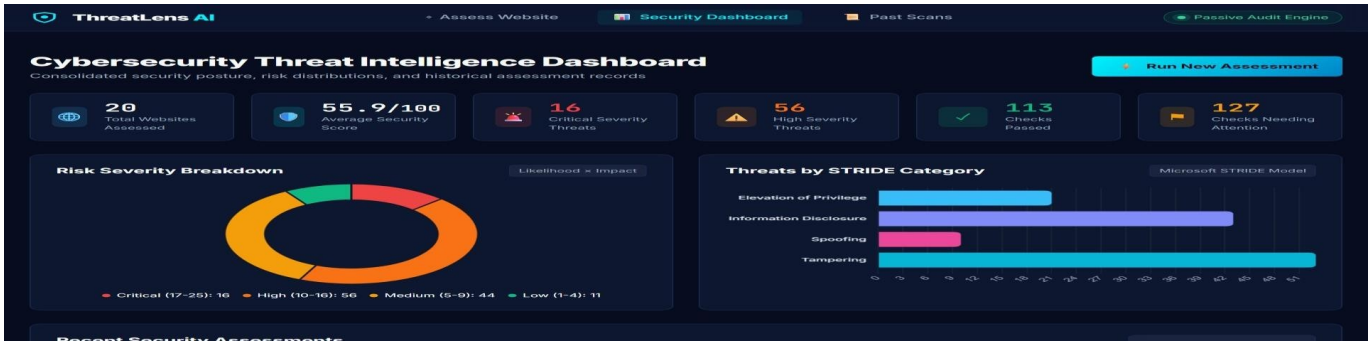



*Figure 2: ThreatLens AI Web Dashboard showing risk scoring, assessment summaries, and monitoring controls (built-in sample data).*

## 8. AI Prompt and AI-Assisted Analysis

The supplied AI prompt asked the assistant to act as a cybersecurity threat-modeling analyst reviewing the ThreatLens AI system. It named four threats: transport security downgrade via cleartext interception, script injection via missing CSP, transparent viewport clickjacking, and server-side request forgery / audit-record tampering. For each threat it requested an attack scenario, impact, practical controls, verification evidence, limitations and a human-review decision, with the constraint that controls must not be claimed as implemented unless verified by code or test output.

AI-Assisted Cybersecurity Threat Analysis Prompt — ThreatLens AI

```
System Prompt: Act as a cybersecurity threat-modeling analyst reviewing the ThreatLens AI ecosystem.
Analyze these four primary threats affecting the web assessment architecture:
  1. Transport Security Downgrade & Cleartext MitM Interception - Tampering / Spoofing (T/S)
  2. Missing Content-Security-Policy & Script Injection (XSS) - Tampering / Elevation of Privilege (T/E)
  3. Transparent Viewport Clickjacking & UI Redressing - Elevation of Privilege (E)
  4. Server-Side Request Forgery & Audit Log Tampering - Tampering / Repudiation (T/R)

For each threat scenario, provide:
  - The attack scenario in clear, beginner-friendly cybersecurity language.
  - The potential impact on confidentiality, integrity, or availability.
  - Practical technical mitigations applicable to client-side and backend logic.
  - The evidence required to verify that the controls are implemented and effective.
  - Limitations of the AI-generated recommendations.
  - A human-review decision: accept recommendations, revise them, or request further evidence.

Constraint: Do not claim that controls are implemented unless verified by empirical code or test output.
```

*Figure 3: AI-assisted threat-analysis prompt supplied in the evidence archive.*

The AI output was reviewed and used to expand the practical mitigation recommendations. Relying solely on client heuristics or basic HTTP checks does not automatically remove the risk; multi-layered controls and human review remain essential.

## 9. Threat 1: Credential Harvesting & Transport Security Downgrade

| | |
|---|---|
| **Affected component** | Transport Layer / Socket & TLS Inspector (security_checks.py) |
| **STRIDE category** | Tampering / Spoofing |
| **Attack scenario** | An attacker on an untrusted local network (e.g. public Wi-Fi) intercepts unencrypted HTTP port 80 traffic or forces an SSL-stripping downgrade to capture credentials and session tokens. |
| **Potential impact** | Loss of confidentiality, cleartext credential theft, unauthorized session hijacking, and reputational damage. |
| **Existing mitigation / status** | Scanner detection logic is implemented. In the knowledge base a failed HTTPS redirect is scored Tampering, Likelihood 4 x Impact 5 = 20 (Critical); a missing HSTS header is scored Likelihood 3 x Impact 3 = 9 (Medium). Target-side remediation is outside the scanner and remains open, so residual risk stays Critical until HTTPS redirection and HSTS are deployed by the target host. |
| **AI recommendations** | Enforce HTTPS redirection and configure HSTS with an appropriate max-age. Enable includeSubDomains only after confirming that all subdomains support HTTPS. Consider HSTS preloading only after meeting the relevant requirements. |
| **Verification evidence** | Automated unit test [PASS] test_downgrade_redirect_detected (mock-based). Figure 4 is a built-in sample scenario, not a live scan. |
| **Human conclusion** | Reasonable recommendations. HSTS helps prevent downgrade attacks after the policy is learned or preloaded, provided HTTPS is correctly configured. Residual risk remains before the first secure visit. |

## 9.1 Threat 1: Updated Mitigation

The updated mitigation recommendations combine syntactic transport inspection with quantitative risk scoring. Under the implemented scoring formula in threat_model.py (Risk Score = Likelihood [1-5] x Impact [1-5]), if a target website fails to redirect plaintext HTTP traffic to HTTPS, the engine flags a Critical Tampering threat (Likelihood 4, Impact 5, Risk Score 20). If an expired or untrusted certificate is detected, the engine generates an independent Critical Spoofing threat record (Likelihood 4, Impact 5, Risk Score 20). An insecure redirect to plaintext HTTP is scored 3 x 5 = 15 (High). Figure 4 shows the built-in "Insecure Legacy Web Portal" sample scenario (hygiene score 22/100) with 2 Critical and 5 High threats. Because this hostname uses the .internal suffix, which the anti-SSRF validator deliberately blocks, it cannot be scanned live. Likelihood and Impact can be adjusted through dropdown selectors on the results page.

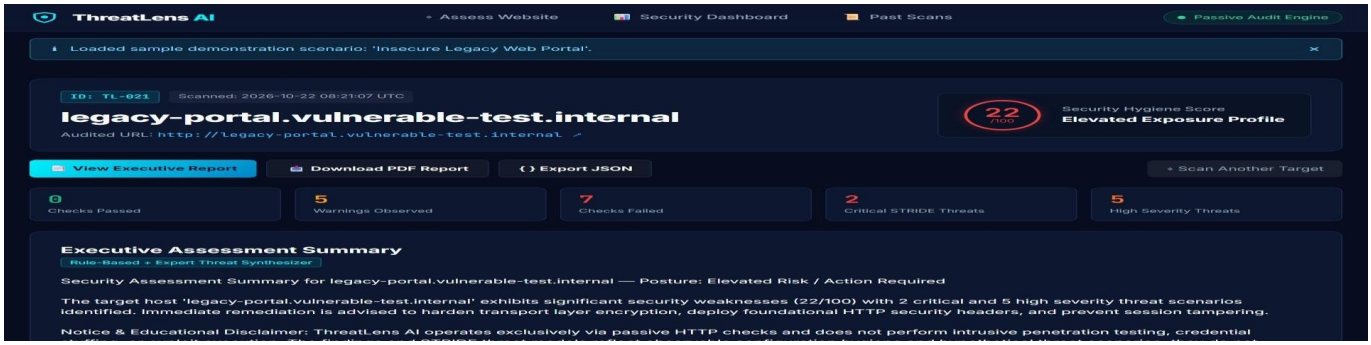



*Figure 4: Threat 1 assessment screen showing transport security warning, evidence breakdown, and safe recommendations (built-in sample scenario).*

security_checks.py — check_tls_certificate & HTTPS Enforcement — Threat 1 Defense Engine

```
# Threat 1 Defense: TLS Certificate & Protocol Cryptography Verification
context = ssl.create_default_context()  # Verification NEVER disabled
with socket.create_connection((hostname, port), timeout=REQUEST_TIMEOUT) as sock:
    with context.wrap_socket(sock, server_hostname=hostname) as ssock:
        cert = ssock.getpeercert()
        tls_version = ssock.version()  # TLSv1.2 or TLSv1.3
        cipher = ssock.cipher()
        days_left = (not_after - datetime.now(timezone.utc)).days
        if days_left < 0:  # Expired cert triggers critical spoofing alert
            result['status'] = 'FAIL'
            result['evidence'] = f'TLS certificate expired {abs(days_left)} days ago.'
        elif days_left <= 30:
            result['status'] = 'WARNING'  # Renewal SLA alert
        else:
            result['status'] = 'PASS'  # Modern cryptography confirmed
```

*Figure 5: Simplified excerpt of the TLS inspection logic in security_checks.py (an expired or untrusted certificate raises SSLCertVerificationError and is recorded as FAIL).*

## 10. Threat 2: Deceptive Script Execution & Missing Content-Security-Policy

| | |
|---|---|
| **Affected component** | Security Headers Inspector (security_checks.py) |
| **STRIDE category** | Tampering / Elevation of Privilege |
| **Attack scenario** | If an input sanitization flaw exists in the target application, the absence of a Content-Security-Policy allows injected JavaScript to execute unrestricted, exfiltrating tokens and modifying DOM elements. |
| **Potential impact** | Cross-site scripting (XSS) amplification, session token theft, unauthorized client-side actions, and telemetry tracking. |
| **Existing mitigation / status** | Scanner check is implemented in security_checks.py. A missing CSP is scored Likelihood 3 x Impact 4 = 12 (High); a permissive CSP (unsafe-inline, unsafe-eval, wildcard) is scored 2 x 3 = 6 (Medium). Target-side mitigation is outside the scanner; residual risk stays High (12) until a strict CSP is deployed. |
| **AI recommendations** | Deploy a strict CSP header with default-src 'self', script nonces, object-src 'none', and a CSP violation reporting endpoint. |
| **Verification evidence** | Automated unit tests [PASS] test_missing_all_security_headers and [PASS] test_permissive_csp_warning. |
| **Human conclusion** | Strict CSP provides valuable defense-in-depth against injection flaws, though contextual output sanitization remains necessary. |

security_checks.py — check_security_headers (CSP & HSTS Auditing) Threat 2 Defense Engine

```
# Threat 2 Defense: Content-Security-Policy & Script Execution Boundary
csp = normalized_headers.get('content-security-policy')
if not csp:
    header_results['Content-Security-Policy'] = {
        'status': 'FAIL',
        'evidence': 'Header missing. Browser does not restrict executable scripts (XSS risk).'
    }
else:
    warnings = []
    if "'unsafe-inline'" in csp: warnings.append("weakens XSS mitigation via inline scripts")
    if "'unsafe-eval'" in csp:   warnings.append("permits string-to-code execution")
    if "default-src *" in csp:   warnings.append("overly permissive wildcard origin")
    status = 'WARNING' if warnings else 'PASS'
    header_results['Content-Security-Policy'] = {'status': status, 'evidence': ', '.join(warnings)}
```

*Figure 6: Simplified excerpt of the Threat 2 HTTP security header audit logic and CSP defense in security_checks.py.*

## 11. Threat 3: Clickjacking & Transparent Viewport Traps

| | |
|---|---|
| **Affected component** | Frame Protection check in check_security_headers() (security_checks.py) |
| **STRIDE category** | Elevation of Privilege (UI redressing is the attack mechanism) |
| **Attack scenario** | An attacker overlays an invisible, full-screen transparent iframe (e.g., opacity: 0, z-index: 999999) on a decoy page so that a user's click triggers an action in the target site while authenticated. |
| **Potential impact** | Involuntary execution of privileged actions, fraudulent transactions, altered user permissions, and UI deception. |
| **Existing mitigation / status** | Scanner detection is implemented. Missing frame protection is scored Likelihood 3 x Impact 4 = 12 (High). Target-side remediation is outside the scanner and remains open until framing restrictions are enforced. |
| **AI recommendations** | Configure Content-Security-Policy: frame-ancestors 'self'; also set X-Frame-Options: SAMEORIGIN; set SameSite=Lax on session cookies. |
| **Verification evidence** | Unit tests test_missing_all_security_headers (FAIL when both headers are absent) and test_well_hardened_security_headers (PASS) confirm scanner detection for the X-Frame-Options path. The CSP frame-ancestors path is verified by code review only. Real browser iframe embedding was not tested. |
| **Human conclusion** | frame-ancestors is the modern standard for clickjacking prevention; X-Frame-Options is a legacy fallback. SameSite cookies provide only secondary protection. |

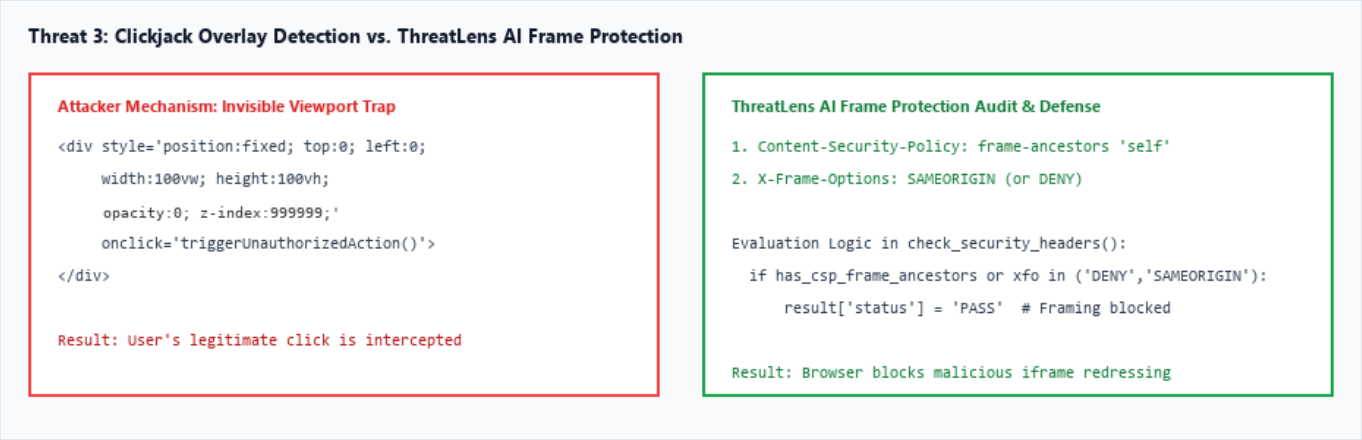



*Figure 7: Frame Protection Header Validation Against Clickjacking (illustrative diagram, not runtime evidence).*

## 12. Threat 4: Request Validation (SSRF) & Audit-Record Integrity

| | |
|---|---|
| **Affected component** | Anti-SSRF URL Pre-Validator & SQLite Database (security_checks.py, database.py, app.py) |
| **STRIDE category** | Information Disclosure (SSRF) / Tampering / Repudiation |
| **Attack scenario** | An attacker submits internal IP targets (127.0.0.1, 169.254.169.254) to make the server send requests into the internal network or cloud metadata service, or injects SQL payloads into the analysis form to alter or truncate stored assessment records. |
| **Potential impact** | Internal cloud metadata exposure, falsification of security audit records, database corruption, and non-repudiation failure. |
| **Existing mitigation / status** | Implemented: anti-SSRF validator with scheme restriction, blocked hostname list, embedded-credential rejection, and socket.getaddrinfo pre-resolution blocking private, loopback, link-local, reserved and cloud-metadata addresses; SSRF re-validation on every redirect hop (safe_http_request calls validate_and_resolve_url before each request, with automatic redirect following disabled); parameterized SQLite queries in database.py; assessment status values restricted to Open, In Progress, Mitigated and Accepted. Not implemented: pinning the resolved IP for the connection, authentication, CSRF protection and an audit trail for the delete and update routes. Residual risk (analyst-estimated, not computed by the tool): Likelihood 2 x Impact 4 = 8 (Medium). |
| **AI recommendations** | Pin the resolved IP for the connection to reduce DNS-rebinding risk (the validator resolves the name once and the HTTP library resolves it again); add authentication and CSRF tokens to state-changing routes; HMAC-SHA256 chaining of history rows; append-only audit log; remote SIEM/syslog replication. |
| **Verification evidence** | Automated unit tests [PASS] test_block_localhost, [PASS] test_block_cloud_metadata_ipv4 and [PASS] test_ssrf_blocking_via_form_post; per-hop redirect re-validation and parameterized SQL queries confirmed by code review (no dedicated test for either; test_redirect_limit_exceeded covers only the 5-hop limit, and there is no dedicated SQL injection test). |
| **Human conclusion** | Pre-connection DNS validation with per-hop redirect re-validation is effective against direct attacks, and parameterized queries address SQL injection, but neither guarantees protection against advanced cases such as DNS rebinding (the resolved IP is not pinned), and neither addresses repudiation. IP pinning, HMAC chaining and SIEM replication remain recommendations until implemented and verified. |

### 12.1 Threat 4 Evidence

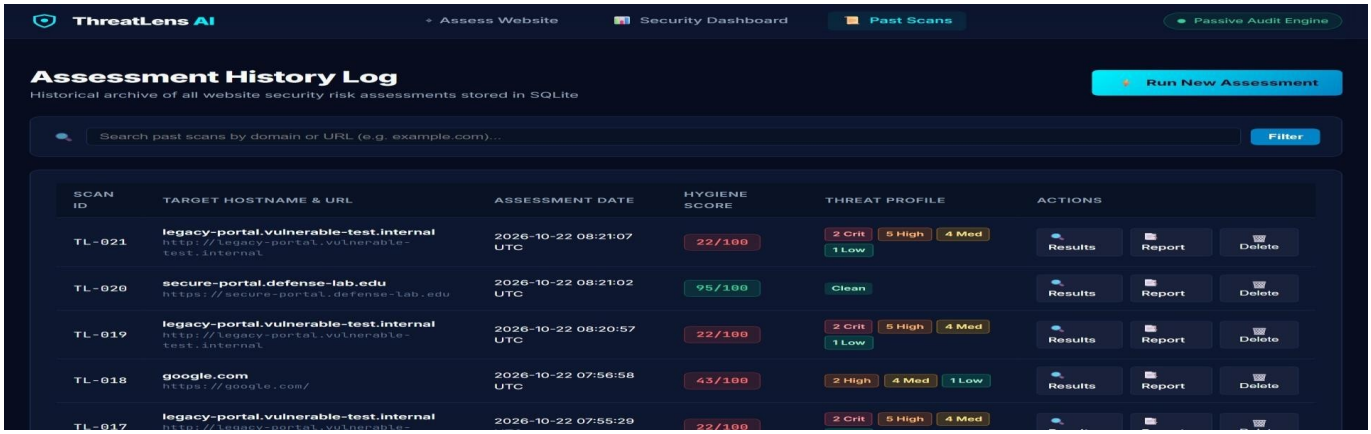



*Figure 8: Flask web analyzer interface showing threat metrics, assessment records, and SQLite analysis ledger.*

## 13. Existing Mitigations versus AI Recommendations

| Threat | Existing evidence | AI-assisted recommendation | Evidence status |
|---|---|---|---|
| Transport Downgrade | HTTPS redirect check (Score 20), HSTS check (Score 9), TLS validity check (Score 20) | HSTS preload, TLS 1.3 protocol enforcement, DNS CAA records | Verified via unit tests (mock-based); no live-socket test; recommendations not implemented |
| Script Injection (CSP) | CSP audit, unsafe-inline & wildcard inspection; Score 12 | Cryptographic nonces, object-src 'none', CSP reporting URI | Verified via response header unit tests |
| Clickjacking | Frame-ancestors & X-Frame-Options inspection; Score 12 | CSP frame-ancestors 'self', SameSite=Lax cookie attribute | Verified via header unit tests; browser iframe test not done |
| SSRF | Anti-SSRF pre-validator with per-hop redirect re-validation | IP pinning to reduce DNS-rebinding risk | Direct SSRF blocking verified by unit tests; per-hop re-validation by code review; DNS rebinding not covered |
| Audit-record integrity | Parameterized SQLite queries; Score 8 (analyst-estimated) | Authentication, CSRF, HMAC chaining, append-only log, SIEM replication | SQL queries confirmed by code review; other items not implemented |

The comparison shows that AI broadened short mitigation statements into defense-in-depth themes. It does not establish that every proposed technology is implemented in code.

## 14. Human Evaluation of AI Recommendations

Human judgment was required to decide whether each AI recommendation matched the threat, was technically reasonable, and went beyond the evidence. The most critical review decision was to keep ThreatLens AI passive, safe, and privacy-preserving, rejecting AI suggestions that required intrusive payload injection or external telemetry servers. A second decision was to maintain a transparent risk score formula (Likelihood x Impact = 1 to 25) instead of an opaque AI-generated score; scores that the tool does not compute (Threat 4) are labelled analyst-estimated. The evaluation also separates a three-stage lifecycle: *Detection* (passive audit checks implemented and passing in unit tests), *Remediation* (the target owner's task; the tool tracks a workflow status such as 'In Progress' in SQLite), and *Verification* (automated tests confirming scanner behaviour, not target security).

AI-generated claims that could not be verified were corrected. For example, a claim that a CSP is sent on all endpoints was rejected because the Flask application itself does not set security headers, and IP pinning was recorded as a recommendation because the code does not implement it.

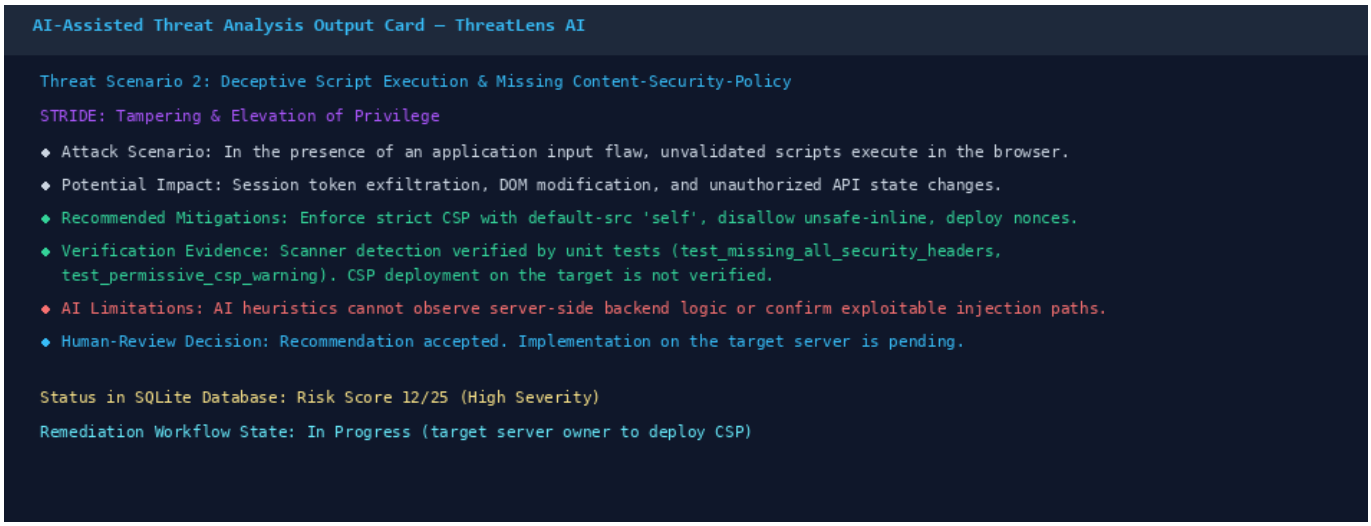



*Figure 9: AI-assisted threat analysis response including limitations and a human-review decision (verification refers to scanner detection only).*

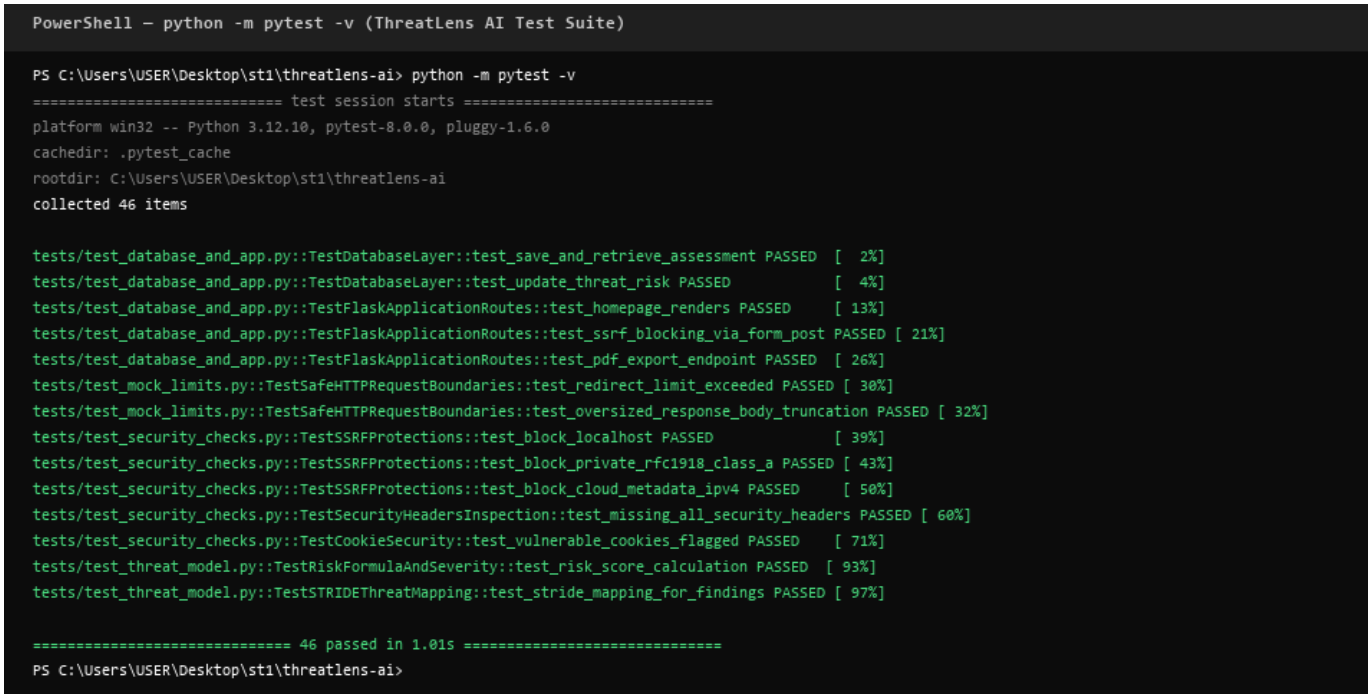



*Figure 10: Automated unit test suite execution confirming 46 passing tests with zero failures.*

## 15. Results and Discussion

- The multi-tier ThreatLens AI architecture was successfully modeled across client, security engine, and server tiers.
- Four threat scenarios were analysed covering Spoofing, Tampering, Information Disclosure, Repudiation, and Elevation of Privilege.
- Threat 1: the scanner detected a missing HTTPS redirect and HSTS, scored Critical (20) and Medium (9) in the sample scenario.
- Threat 2: the scanner audits missing and permissive CSP; a missing CSP scores High (12).
- Threat 3: frame-protection detection scores High (12) and is verified by unit tests for the X-Frame-Options path (test_missing_all_security_headers, test_well_hardened_security_headers); the CSP frame-ancestors path is verified by code review only.
- Threat 4: the anti-SSRF validator is verified by test_block_localhost, test_block_cloud_metadata_ipv4 and test_ssrf_blocking_via_form_post; per-hop redirect re-validation and SQL injection resistance rest on code review. The app has no authentication, CSRF protection or audit trail.
- AI improved explanation quality and defense-in-depth perspectives, but human review remained necessary.

**Result: STRIDE threat modeling and AI-assisted analysis were applied to four threat scenarios in ThreatLens AI, and AI recommendations were reviewed against code and test evidence. Only the detection logic and the anti-SSRF validator are verified by tests; IP pinning, HMAC chaining, SIEM replication, CSP nonces, HSTS preloading and CAA records remain recommendations.**

## 16. Limitations and Recommendations

### 16.1 Limitations

- Passive HTTP/TLS checks cannot inspect internal server-side application logic, database queries, or stored credentials.
- Web applications with client-side JavaScript rendering (SPAs) may dynamically load content that static HTML analysis cannot observe.
- Redirect analysis is limited to 5 hops, a 6-second timeout and a 2 MB response limit.
- The dashboard figures use built-in sample data; sample hostnames use .internal and cannot be scanned live.

- There is no live-socket TLS test and no real browser iframe embedding test.
- The Flask app has no authentication, CSRF protection or security headers, runs in debug mode and has a default fallback secret key; it is intended for local use (127.0.0.1) only.
- AI recommendations can introduce unnecessary complexity if accepted without scrutiny.

### 16.2 Recommendations

- Add authentication and CSRF tokens to the delete and update routes, and disable debug mode and the default secret key outside local use.
- Add an append-only audit log with HMAC-SHA256 chaining for assessment history.
- Add security headers (CSP, X-Frame-Options, X-Content-Type-Options) to the Flask application itself.
- Add a mocked live-TLS test and a browser iframe embedding test for end-to-end validation.
- Pin resolved IPs for the connection to reduce DNS-rebinding risk, and add a test for per-hop redirect re-validation.
- Add a dedicated SQL injection test and expand test fixtures to cover unicode homograph (punycode) hostnames.

## 17. Conclusion

This project applied the STRIDE threat modeling methodology and AI-assisted analysis to four threat scenarios in the ThreatLens AI cybersecurity ecosystem. AI expanded baseline mitigation ideas into structured, multi-tier controls covering transport security, script injection boundaries, clickjacking, and request validation and record integrity. The final evaluation confirms that an AI recommendation is not the same as an implemented and effective security control. Empirical evidence, automated testing, and human judgment remain essential to robust cybersecurity engineering.

## 18. References


1. OWASP Threat Modeling: https://owasp.org/www-community/Threat_Modeling
2. Microsoft STRIDE Threat Modeling: https://learn.microsoft.com/en-us/azure/security/develop/threat-modeling-tool-threats
3. OWASP Secure Headers Project: https://owasp.org/www-project-secure-headers/
4. OWASP Top 10 Web Application Security Risks: https://owasp.org/www-project-top-ten/
5. OWASP Application Security Verification Standard: https://owasp.org/www-project-application-security-verification-standard/
6. IETF RFC 7034 (X-Frame-Options): https://datatracker.ietf.org/doc/html/rfc7034
7. IETF RFC 6797 (HTTP Strict Transport Security): https://datatracker.ietf.org/doc/html/rfc6797
8. IETF RFC 1918 (Address Allocation for Private Internets): https://datatracker.ietf.org/doc/html/rfc1918